Module – 5

Pure React – Diving Deeper into React

The Virtual DOM

The React virtual DOM (VDOM) is a programming concept where a lightweight, in-memory representation of the actual DOM is kept in memory and synchronized with the real DOM only when necessary. This process significantly optimizes performance by minimizing costly direct manipulations of the real DOM.

How the Virtual DOM Works

When an application's state changes, React follows a specific process to update the user interface efficiently:

1. Virtual DOM Updated: When a component's state or props change, React creates a new virtual DOM tree, which is a plain JavaScript object that represents the new UI state.
2. Diffing: React then compares this new virtual DOM tree with a snapshot of the previous virtual DOM using a process called "diffing". This algorithm identifies exactly what has changed in the UI.
3. Reconciliation & Batching: After determining the differences, React calculates the minimal set of changes needed and batches these updates into a single operation.
4. Real DOM Update: Finally, only the changed parts are updated in the actual, on-screen DOM. This is handled by the ReactDOM library.

Key Benefits

1. Performance Optimization: Manipulating the real DOM is slow due to the browser's required reflow and repaint processes. By only updating the necessary parts, the virtual DOM significantly reduces this overhead.
2. Declarative UI: The VDOM allows developers to use a declarative programming style—describing how the UI should look in a given state—while React handles the complex imperative steps of updating the actual DOM to match that state.
3. Abstraction: It abstracts away the complexity of manual DOM manipulation, making code more predictable, readable, and easier to maintain.

Virtual DOM vs. Real DOM

Feature	Real DOM	Virtual DOM
Description	The actual tree structure of the web page's HTML elements.	A lightweight, in-memory replica of the actual DOM.
Manipulation	Directly manipulates on-screen elements; can be slow for frequent updates.	Does not directly manipulate the screen; uses diffing and batching for fast updates.
Performance	Slower updates due to full tree re-renders and repaints.	Faster updates by applying only minimal, batched changes.
Used by	All web browsers (a native web standard).	Libraries and frameworks like React, Vue.js, etc..

React Elements

The browser DOM is made up of DOM elements. Similarly, the React DOM is made up of React elements. DOM elements and React elements may look the same, but they are actually quite different. A React element is a description of what the actual .DOM element should look like. In other words, React elements are the instructions for how the browser DOM should be created.

We can create a React element to represent an h1 using React.createElement:

```
React.createElement("h1", null, "Baked Salmon")
```

The first argument defines the type of element that we wish to create. In this case, we want to create a heading-one element. The third argument represents the element's children, any nodes that are inserted between the opening and closing tag. The second argument represents the element's properties. This h1 currently does not have any properties.

During rendering, React will convert this element to an actual DOM element:

```
<h1>Baked Salmon</h1>
```

When an element has attributes, they can be described with properties. Here is a sample of an HTML h1 tag that has id and data-type attributes:

```
React.createElement("h1",
{id: "recipe-0", 'data-type': "title"},
"Baked Salmon"
)
```

```
<h1 data-reactroot id="recipe-0" data-type="title">Baked Salmon</h1>
```

React DOM

ReactDOM contains the tools necessary to render React elements in the browser. ReactDOM is where we will find the render method as well as the renderToString and renderToStaticMarkup methods that are used on the server.

We can render a React element, including its children, to the DOM with ReactDOM.render. The element that we wish to render is passed as the first argument and the second argument is the target node, where we should render the element:

```
var dish = React.createElement("h1", null, "Baked Salmon")

ReactDOM.render(dish, document.getElementById('react-container'))
```

Rendering the title element to the DOM would add a heading-one element to the div with the id of react-container, which we would already have defined in our HTML.

In Example , we build this div inside the body tag.

Example : React added the h1 element to the target: react-container

```
<body>

<div id="react-container">

<h1>Baked Salmon</h1>

</div>

</body>
```

All of the DOM rendering functionality in React has been moved to ReactDOM because we can use React to build native applications as well. The browser is just one target for React. That's all you need to do. You create an element, and then you render it to the DOM.

Children

ReactDOM allows you to render a single element to the DOM.6 React tags this as data-reactroot. All other React elements are composed into a single element using nesting.

React renders child elements using props.children. In the previous section, we rendered a text element as a child of the h1 element, and thus props.children was set to "Baked Salmon". We could render other React elements as children too, creating at tree of elements. This is why we use the term component tree. The tree has one root component from which many branches grow.

Let's consider the unordered list that contains ingredients in Example.

Example : Ingredients list

```
<ul>

<li>1 lb Salmon</li>

<li>1 cup Pine Nuts</li>

<li>2 cups Butter Lettuce</li>
```

1 Yellow Squash

1/2 cup Olive Oil

3 cloves of Garlic

In this sample, the unordered list is the root element, and it has six children. We can represent this ul and its children with React.createElement (Example).

Example : Unordered list as React elements

```
React.createElement(
"ul",
null,
React.createElement("li", null, "1 lb Salmon"),
React.createElement("li", null, "1 cup Pine Nuts"),
React.createElement("li", null, "2 cups Butter Lettuce"),
React.createElement("li", null, "1 Yellow Squash"),
React.createElement("li", null, "1/2 cup Olive Oil"),
React.createElement("li", null, "3 cloves of Garlic")
)
```

Every additional argument sent to the createElement function is another child element. React creates an array of these child elements and sets the value of props.children to that array.

If we were to inspect the resulting React element, we would see each list item represented by a React element and added to an array called props. Children.

Example : Resulting React element

```
{
"type": "ul",
"props": {
"children": [
{ "type": "li", "props": { "children": "1 lb Salmon" } … },
{ "type": "li", "props": { "children": "1 cup Pine Nuts"} … },
{ "type": "li", "props": { "children": "2 cups Butter Lettuce" } … },
{ "type": "li", "props": { "children": "1 Yellow Squash"} … },
{ "type": "li", "props": { "children": "1/2 cup Olive Oil"} … },
{ "type": "li", "props": { "children": "3 cloves of Garlic"} … }
]
```

```
...
}
}
```

We can now see that each list item is a child. Earlier in this chapter, we introduced HTML for an entire recipe rooted in a section element. To create this using React, we'll use a series of createElement calls, as in

Example . React Element tree

```
React.createElement("section", {id: "baked-salmon"},
React.createElement("h1", null, "Baked Salmon"),
React.createElement("ul", {"className": "ingredients"},
React.createElement("li", null, "1 lb Salmon"),
React.createElement("li", null, "1 cup Pine Nuts"),
React.createElement("li", null, "2 cups Butter Lettuce"),
React.createElement("li", null, "1 Yellow Squash"),
React.createElement("li", null, "1/2 cup Olive Oil"),
React.createElement("li", null, "3 cloves of Garlic")
),
React.createElement("section", {"className": "instructions"},
React.createElement("h2", null, "Cooking Instructions"),
React.createElement("p", null, "Preheat the oven to 350 degrees."),
React.createElement("p", null,
"Spread the olive oil around a glass baking dish."),
React.createElement("p", null, "Add the salmon, garlic, and pine..."),
React.createElement("p", null, "Bake for 15 minutes."),
React.createElement("p", null, "Add the yellow squash and put..."),
React.createElement("p", null, "Remove from oven and let cool for 15 ....")
)
)
```

This sample is what pure React looks like. Pure React is ultimately what runs in the browser. The virtual DOM is a tree of React elements all stemming from a single root element. React elements are the instructions that React will use to build a UI in the browser.

Constructing Elements with Data

The major advantage of using React is its ability to separate data from UI elements.Since React is just JavaScript, we can add JavaScript logic to help us build the React component tree. For example, ingredients can be stored in an array, and we can map that array to the React elements.

Example Unordered list

```
React.createElement("ul", {"className": "ingredients"},
React.createElement("li", null, "1 lb Salmon"),
React.createElement("li", null, "1 cup Pine Nuts"),
React.createElement("li", null, "2 cups Butter Lettuce"),
React.createElement("li", null, "1 Yellow Squash"),
React.createElement("li", null, "1/2 cup Olive Oil"),
React.createElement("li", null, "3 cloves of Garlic")
);
```

The data used in this list of ingredients can be easily represented using a JavaScriptarray.

Example . items array

```
var items = [
"1 lb Salmon",
"1 cup Pine Nuts",
"2 cups Butter Lettuce",
"1 Yellow Squash",
"1/2 cup Olive Oil",
"3 cloves of Garlic"
]
```

We could construct a virtual DOM around this data using the Array.map function, as in Example . Mapping an array to li elements

```
React.createElement(
"ul",
{ className: "ingredients" },
items.map(ingredient =>
React.createElement("li", null, ingredient)
)
)
```

This syntax creates a React element for each ingredient in the array. Each string is displayed in the list item's children as text. The value for each ingredient is displayed as the list item.

When we build a list of child elements by iterating through an array, React likes each of those elements to have a key property. The key property is used by React to help it update the DOM efficiently. We will be discussing keys and why we need them in but for now you can make this warning go away by adding a unique key property to each of the list item elements .We can use the array index for each ingredient as that unique value.

Example . Adding a key property

```
React.createElement("ul", {className: "ingredients"},
items.map((ingredient, i) =>
React.createElement("li", { key: i }, ingredient))
```

React Components

In React, components are reusable, independent code blocks (A function or a class) that define the structure and behavior of the UI. They accept inputs (props or properties) and return elements that describe what should appear on the screen.

Key Concepts of React Components:

- Each component handles its own logic and UI rendering.
- Components can be reused throughout the app for consistency.
- Components accept inputs via props and manage dynamic data using state.
- Only the changed component re-renders, not the entire page.

Example :

```
import React from 'react';
// Creating a simple functional component
function Greeting() {
  return (
    <h1>Hello, welcome to React!</h1>
  );
}
export default Greeting;
```

Output :

Hello, welcome to React!

Types of React Components

There are two primary types of React components:

1. Functional Components

Functional components are simpler and preferred for most use cases. They are JavaScript functions that return React elements. With the introduction of React Hooks, functional components can also manage state and lifecycle events.

- **Stateless or Stateful:** Can manage state using React Hooks.
- **Simpler Syntax:** Ideal for small and reusable components.
- **Performance:** Generally faster since they don't require a 'this' keyword.

Example :

```
function
Greet(props){
   return<h1>Hello,{props.name}!</h1>;
}
```

2. Class Components

Class components are ES6 classes that extend React.Component. They include additional features like state management and lifecycle methods.

- **State Management:** State is managed using the this.state property.
- **Lifecycle Methods:** Includes methods like componentDidMount,componentDidUpdate, etc.

Example:

```
class Greet extends React.Component
{
   render(){
      return<h1>Hello,{this.props.name}!</h1>;
  }
}
```

CreateClass Method

When React was first introduced in 2013, there was only one way to create a component: the createClass function

New methods of creating components have emerged, but createClass is still used widely in React projects. The React team has indicated, however, that createClass may be deprecated in the future.

Example Ingredients list as a React component

```
const IngredientsList = React.createClass({

displayName: "IngredientsList",

render() {

return React.createElement("ul", {"className": "ingredients"},

React.createElement("li", null, "1 lb Salmon"),

React.createElement("li", null, "1 cup Pine Nuts"),

React.createElement("li", null, "2 cups Butter Lettuce"),
```

```
React.createElement("li", null, "1 Yellow Squash"),
React.createElement("li", null, "1/2 cup Olive Oil"),
React.createElement("li", null, "3 cloves of Garlic")
)
}
})
const list = React.createElement(IngredientsList, null, null)
ReactDOM.render(
list,
document.getElementById('react-container')
)
```

Components allow us to use data to build a reusable UI. In the render function, we can use the this keyword to refer to the component instance, and properties can be accessed on that instance with this.props.

Here, we have created an element using our component and named it Ingredients

List:

```
<IngredientsList>
<ul className="ingredients">
<li>1 lb Salmon</li>
<li>1 cup Pine Nuts</li>
<li>2 cups Butter Lettuce</li>
<li>1 Yellow Squash</li>
<li>1/2 cup Olive Oil</li>
<li>3 cloves of Garlic</li>
</ul>
</IngredientsList>
```

Data can be passed to React components as properties. We can create a reusable list ofingredients by passing that data to the list as an array:

```
const IngredientsList = React.createClass({
displayName: "IngredientsList",
render() {
return React.createElement("ul", {className: "ingredients"},
this.props.items.map((ingredient, i) =>
React.createElement("li", { key: i }, ingredient)
```

```
)
)
}
})
const items = [
"1 lb Salmon",
"1 cup Pine Nuts",
"2 cups Butter Lettuce",
"1 Yellow Squash",
"1/2 cup Olive Oil",
"3 cloves of Garlic"
]
ReactDOM.render(
React.createElement(IngredientsList, {items}, null),
document.getElementById('react-container')
)
```

Now, let's look at ReactDOM. The data property items is an array with six ingredients. Because we made the li tags using a loop, we were able to add a unique key using the index of the loop:

```
<IngredientsList items=[...]>
<ul className="ingredients">
<li key="0">1 lb Salmon</li>
<li key="1">1 cup Pine Nuts</li>
<li key="2">2 cups Butter Lettuce</li>
<li key="3">1 Yellow Squash</li>
<li key="4">1/2 cup Olive Oil</li>
<li key="5">3 cloves of Garlic</li>
</ul>
</IngredientsList>
```

The components are objects. They can be used to encapsulate code just like classes. We can create a method that renders a single list item and use that to build out the list

Example : With a custom method

```
const IngredientsList = React.createClass({
```

```
displayName: "IngredientsList",
React Components | 71renderListItem(ingredient, i) {
return React.createElement("li", { key: i }, ingredient)
},
render() {
return React.createElement("ul", {className: "ingredients"},
this.props.items.map(this.renderListItem)
)
}
})
```

ES6 Classes

One of the key features included in the ES6 spec are classes. React.Component is an abstract class that we can use to build new React components. We can create custom components through inheritance by extending

This class with ES6 syntax. We can create IngredientsList using the same syntax

Example. IngredientsList as an ES6 class

```
class IngredientsList extends React.Component {
renderListItem(ingredient, i) {
return React.createElement("li", { key: i }, ingredient)
}
render() {
return React.createElement("ul", {className: "ingredients"},
this.props.items.map(this.renderListItem)
)
}
}
```

Stateless Functional Components

Stateless functional components are functions, not objects; therefore, they do not have a “this” scope. Because they are simple, pure functions, we’ll use them as much as possible in our applications. There may come a point where the stateless functional component isn’t robust enough and we must fall back to using class or createClass, but in general the more you can use these, the better. Stateless functional components are functions that take in properties and return aDOM element.

Stateless functional components are a good way to pr actice the rulesof functional programming. You should strive to make each stateless functional component a pure function. They should take in props and return a DOM element without causing side effects. This encourages simplicity and makes the codebase extremely testable.

Stateless functional components will keep your application architecture simple, and The React team promises some performance gains by using them. If you need to encapsulate functionality or have a this scope, however, you can't use them.

In example we combine the functionality of renderListItem and render into a single function.

Example : Creating a stateless functional component

```
const IngredientsList = props =>
React.createElement("ul", {className: "ingredients"},
props.items.map((ingredient, i) =>
React.createElement("li", { key: i }, ingredient)
)
```

Using ES6 destructuring syntax, we can scope the list property directly to this function, reducing the repetitive dot syntax. Now we'd use the IngredientsList the same way we render component classes.

Example . Destructuring the properties argument

```
const IngredientsList = ({items}) =>
React.createElement("ul", {className: "ingredients"},
items.map((ingredient, i) =>
React.createElement("li", { key: i }, ingredient)
)
)
)
```

DOM Rendering

In React.js, DOM rendering is a highly efficient, multi-step process managed by the library using a Virtual DOM (VDOM). This approach minimizes direct interactions with the actual browser DOM to optimize performance.

The Rendering Process

The entire process occurs in three main steps:

1. Trigger a Render: A render is initiated when the application first starts (initial render) or when a component's state or props are updated.
2. Render Components: React calls the relevant components to determine what should be displayed on the screen. This results in a new, in-memory representation called the Virtual DOM. React uses a "diffing" algorithm to compare this new VDOM tree with the previous one, calculating the minimal necessary changes.
3. Commit Changes to the DOM: In this final "commit" phase, React applies only the identified changes to the actual browser DOM. This targeted updating prevents unnecessary repaints of the entire page, providing a smooth and responsive user experience.

Key Concepts

- Virtual DOM (VDOM): A lightweight JavaScript object that is a representation of the real DOM. React modifies the VDOM first, which is much faster than manipulating the real DOM directly.
- Reconciliation: The process where React compares the old and new VDOMs to figure out which parts of the real DOM need to be updated. This ensures that only the necessary DOM nodes are modified.
- ReactDOM: A package that provides DOM-specific methods. The createRoot()function (used in React 18+) is the modern way to define the top-level DOM element (the "root" node) that React manages, replacing the legacy ReactDOM.render().
- JSX: A syntax extension that allows developers to write HTML-like code within JavaScript, which is then transpiled into React elements (plain JavaScript objects) that React uses to build the VDOM.

Initial Render Example (React 18+)

To initially render your application, you specify a root DOM node (usually a in your index.html) and use the createRoot API:

```
import { createRoot } from 'react-dom/client';
import App from './App.js'; // Your main component

const container = document.getElementById('root');
const root = createRoot(container); // Create a root
root.render(<App />); // Render your component inside the root
```

From this point on, React manages the DOM inside the 'root' container, and updates are handled automatically through state and prop changes, without manually calling render again.

Factories

So far, the only way we have created elements has been with React.createElement. Another way to create a React element is to use factories. A factory is a special object that can be used to abstract away the details of instantiating objects. In React, we use factories to help us create React element instances.

React has built-in factories for all commonly supported HTML and SVG DOM elements, and you can use the React.createFactory function to build your own factories around specific components.

For example, consider our h1 element from earlier in this chapter:

```
<h1>Baked Salmon</h1>
```

Instead of using createElement, we can create a React element with a built-in factory

Example . Using createFactory to create an h1

```
React.DOM.h1(null, "Baked Salmon")
```

In this case, the first argument is for the properties and the second argument is for the children. We can also use DOM factories to build an unordered list, as in

Example : Building an unordered list with DOM factories

```
React.DOM.ul({"className": "ingredients"},
React.DOM.li(null, "1 lb Salmon"),
React.DOM.li(null, "1 cup Pine Nuts"),
React.DOM.li(null, "2 cups Butter Lettuce"),
React.DOM.li(null, "1 Yellow Squash"),
React.DOM.li(null, "1/2 cup Olive Oil"),
React.DOM.li(null, "3 cloves of Garlic")
)
```

In this case, the first argument is for the properties, where we define the className. Additional arguments are elements that will be added to the children array of the unordered list. We can also separate out the ingredient data and improve the preceding definition using factories (Example 4-20).

Example : Using map with factories

```
var items = [
"1 lb Salmon",
"1 cup Pine Nuts",
"2 cups Butter Lettuce",
"1 Yellow Squash",
"1/2 cup Olive Oil",
"3 cloves of Garlic"
]
var list = React.DOM.ul(
{ className: "ingredients" },
items.map((ingredient, key) =>
React.DOM.li({key}, ingredient)
)
)
ReactDOM.render(
list,
document.getElementById('react-container')
)
```

Using Factories with Components

If you would like to simplify your code by calling components as functions, you need to explicitly create a factory.

Example : Creating a factory with IngredientsList

```
const { render } = ReactDOM;
const IngredientsList = ({ list }) =>
React.createElement('ul', null,
list.map((ingredient, i) =>
React.createElement('li', {key: i}, ingredient)
)
)
const Ingredients = React.createFactory(IngredientsList)
const list = [
"1 lb Salmon",
"1 cup Pine Nuts",
"2 cups Butter Lettuce",
"1 Yellow Squash",
"1/2 cup Olive Oil",
"3 cloves of Garlic"
]
render(
Ingredients({list}),
document.getElementById('react-container')
)
```

In this example, we can quickly render a React element with the Ingredients factory. Ingredients is a function that takes in properties and children as arguments just like the DOM factories.

If you are not working with JSX, you may find using factories preferable to numerous React.createElement calls. However, the easiest and most common way to define . React elements is with JSX tags. If you use JSX with React, chances are you will never use a factory.

Props, State, and the Component Tree

Property Validation

JavaScript is a loosely typed language, which means that the data type of a variable's value can change. For example, you can initially set a JavaScript variable as a string,then change its value to an array later, and JavaScript will not complain. Managing our variable types inefficiently can lead to a lot of time spent debugging applications.

React components provide a way to specify and validate property types. Using thisfeature will greatly reduce the amount of time spent debugging applications. Supplying incorrect property types triggers warnings that can help us find bugs that may have otherwise slipped through the cracks.

React has built-in automatic property validation for the variable types, as shown in Table below ,

Type	Validator
Arrays	React.PropTypes.array
Boolean	React.PropTypes.bool
Functions	React.PropTypes.func
Numbers	React.PropTypes.number
Objects	React.PropTypes.object
Strings	React.PropTypes.string

Table : React property validation

Example:

```
Summary.propTypes={
  title:PropTypes.string,
  ingredients:PropTypes.array,
  steps:PropTypes.array
}
```

Required Props

ingredients:PropTypes.array.isRequired

Default Props

Used when values are missing:

```
Summary.defaultProps={
  ingredients:0,
  steps:0,
  title:"[untitled]"
}
```

Custom Prop Validation

Useful when built-in validators are not enough.

Example: Title must be a string and < 20 characters:

```
title: (props, propName) =>
  typeof props[propName] !== "string"
    ? new Error("Title must be a string")
    : props[propName].length > 20
      ? new Error("Title too long")
      : null
```

Using propTypes in ES6 Classes / Functional Components

```
class Summary extends Component {}

Summary.propTypes = {
  // add your prop validations here
};

Summary.defaultProps = {
  // add your default values here
};
```

Functional Components:

```
const Summary = ({ ingredients=0 }) => {}
```

Refs :

References, or refs, are a feature that allow React components to interact with child elements. The most common use case for refs is to interact with UI elements that collect input from the user. Consider an HTML form element. These

elements are initially rendered, but the users can interact with them. When they do, the component should respond appropriately.

Example : AddColorForm

```
import { Component } from 'react'
class AddColorForm extends Component {
render() {
return (
<form onSubmit={e=>e.preventDefault()}>
<input type="text"
placeholder="color title..." required/>
<input type="color" required/>
<button>ADD</button>
</form>
)
}
}
```

The AddColorForm component renders an HTML form that contains three elements : a text input for the title, a color input for the color's hex value, and a button to submit the form. When the form is submitted, a handler function is invoked where the default form event is ignored. This prevents the form from trying to send a GET request once submitted.

Once we have the form rendered, we need to provide a way to interact with it. Specifically, when the form is first submitted, we need to collect the new color information and reset the form's fields so that the user can add more colors. Using refs, we can refer to the title and color elements and interact with them.

Example . AddColorForm with submit method

```
import { Component } from 'react'
class AddColorForm extends Component {
constructor(props) {
super(props)
this.submit = this.submit.bind(this)
}
submit(e) {
const { _title, _color } = this.refs
e.preventDefault();
alert(`New Color: ${_title.value} ${_color.value}`)
_title.value = '';
```

```
_color.value = '#000000';
_title.focus();
}
render() {
return (
<form onSubmit={this.submit}>
<input ref="_title"
type="text"
placeholder="color title..." required/>
<input ref="_color"
type="color" required/>
<button>ADD</button>
</form>
)
}
}
```

Inverse Data Flow

A common solution for collecting data from a React component is inverse data flow. It is similar to, and sometimes described as, two-way data binding. It involves sending a callback function to the component as a property that the component can use to pass data back as arguments. It's called inverse data flow because we send the component a function as a property, and the component sends data back as function arguments.

Let's say we want to use the color form, but when a user submits a new color we want to collect that information and log it to the console.

We can create a function called logColor that receives the title and color as arguments. The values of those arguments can be logged to the console. When we use the AddColorForm, we simply add a function property for onNewColor and set it to our logColor function. When the user adds a new color, logColor is invoked, and we've sent a function as a property:

```
const logColor = (title, color) =>
console.log(`New Color: ${title} | ${value}`)
<AddColorForm onNewColor={logColor} />
```

To ensure that data is flowing properly, we will invoke onNewColor from props with

the appropriate data:

```
submit() {
const {_title, _color} = this.refs
this.props.onNewColor(_title.value, _color.value)
```

```
_title.value =
''
_color.value = '#000000'
_title.focus()
}
```

In our component, this means that we'll replace the alert call with a call to this.props.onNewColor and pass the new title and color values that we have obtained through refs. The role of the AddColorForm component is to collect data and pass it on. It is not concerned with what happens to that data. We can now use this form to collect color data from users and pass it on to some other component or method to handle the collected data:

```
<AddColorForm onNewColor={(title, color) => {
console.log(`TODO: add new ${title} and ${color} to the list`)
console.log(`TODO: render UI with new Color`)
}} />
```

Refs in Stateless Functional Components

Refs can also be used in stateless functional components. These components do not have this, so it's not possible to use this.refs. Instead of using string attributes, we will set the refs using a function. The function will pass us the input instance as an argument. We can capture that instance and save it to a local variable.

Let's refactor AddColorForm as a stateless functional component:

```
const AddColorForm = ({onNewColor=f=>f}) => {
let _title, _color
const submit = e => {
e.preventDefault()
onNewColor(_title.value, _color.value)
_title.value =
''
_color.value = '#000000'
_title.focus()
}
return (
<form onSubmit={submit}>
<input ref={input => _title = input}
type="text"
```

```
placeholder="color title..." required/>
<input ref={input => _color = input}
type="color" required/>
<button>ADD</button>
</form>
)
}
```

In this stateless functional component, refs are set with a callback function instead of a string. The callback function passes the element's instance as an argument. This instance can be captured and saved into a local variable like _title or _color. Once we've saved the refs to local variables, they are easily accessed when the form is submitted.

React State Management

Thus far we've only used properties to handle data in React components. Properties are immutable. Once rendered, a component's properties do not change. In order for our UI to change, we would need some other mechanism that can rerender the component tree with new properties. React state is a built-in option for managing data that will change within a component.

When application state changes, the UI is rerendered to reflect those changes.Users interact with applications. They navigate, search, filter, select, add, update, and delete. When a user interacts with an application, the state of that application changes, and those changes are reflected back to the user in the UI. Screens and menus appear and disappear. Visible content changes. Indicators light up or are turned off. In React, the UI is a reflection of application state.

State can be expressed in React components with a single JavaScript object. When the state of a component changes, the component renders a new UI that reflects those changes. What can be more functional than that? Given some data, a React component will represent that data as the UI. Given a change to that data, React will update the UI as efficiently as possible to reflect that change.

Introducing Component State

State represents data that we may wish to change within a component. To demonstrate this, we will take a look at a StarRating component.

Figure . : The StarRating component

The StarRating component requires two critical pieces of data: the total number of stars to display, and the rating, or the number of stars to highlight.

We'll need a clickable Star component that has a selected property. A stateless functional component can be used for each star:

```
const Star = ({ selected=false, onClick=f=>f }) =>
<div className={(selected) ? "star selected" : "star"}
onClick={onClick}>
</div>
Star.propTypes = {
selected: PropTypes.bool,
onClick: PropTypes.func
}
```

Every Star element will consist of a div that includes the class 'star'. If the star is selected, it will additionally add the class 'selected'. This component also has an optional onClick property. When a user clicks on any star div, the onClick property will be invoked. This will tell the parent component, the StarRating, that a Star has been clicked.

The Star is a stateless functional component. It says it right in the name: you cannot use state in a stateless functional component. Stateless functional components are meant to be the children of more complex, stateful components. It's a good idea to try to keep as many of your components as possible stateless.

Now that we have a Star, we can use it to create a StarRating. StarRating will obtain the total number of stars to display from the component's properties. The rating, the value that the user can change, will be stored in the state.

First, let's look at how to incorporate state into a component defined with create

Class:

```
const StarRating = createClass({
displayName: 'StarRating',
propTypes: {
totalStars: PropTypes.number
},
getDefaultProps() {
eturn {
totalStars: 5
}
},
getInitialState() {
return {
```

```
starsSelected: 0
}
},
change(starsSelected) {
this.setState({starsSelected})
},
render() {
const {totalStars} = this.props
const {starsSelected} = this.state
return (
<div className="star-rating">
{[...Array(totalStars)].map((n, i) =>
<Star key={i}
selected={i<starsSelected}
onClick={() => this.change(i+1)}
/>
)}
<p>{starsSelected} of {totalStars} stars</p>
</div>
)
}
})
```

Initializing State from Properties

We can initialize our state values using incoming properties. There are only a few necessary cases for this pattern. The most common case for this is when we create a reusable component that we would like to use across applications in different component trees.

When using createClass, a good way to initialize state variables based on incoming properties is to add a method called componentWillMount. This method is invoked once when the component mounts, and you can call this.setState() from this method. It also has access to this.props, so you can use values from this.props to help you initialize state:

```
const StarRating = createClass({
displayName: 'StarRating',
propTypes: {
```

```
totalStars: PropTypes.number
},
getDefaultProps() {
return {
totalStars: 5
}
},
getInitialState() {
return {
starsSelected: 0
}
},
componentWillMount() {
const { starsSelected } = this.props
if (starsSelected) {
this.setState({starsSelected})
}
},
change(starsSelected) {
this.setState({starsSelected})
},
render() {
const {totalStars} = this.props
const {starsSelected} = this.state
return (
<div className="star-rating">
{[...Array(totalStars)].map((n, i) =>
<Star key={i}
selected={i<starsSelected}
onClick={() => this.change(i+1)}
/>
```

```
)}
<p>{starsSelected} of {totalStars} stars</p>
</div>
)
}
})
render(
<StarRating totalStars={7} starsSelected={3} />,
document.getElementById('react-container')
)
```

componentWillMount is a part of the component lifecycle. It can be used to help you initialize state based on property values in components created with createClass or ES6 class components.

There is an easier way to initialize state within an ES6 class component. The constructor receives properties as an argument, so you can simply use the props argument passed to the constructor:

```
constructor(props) {
super(props)
this.state = {
starsSelected: props.starsSelected || 0
}
this.change = this.change.bind(this)
}
```

For the most part, you'll want to avoid setting state variables from properties. Only use these patterns when they are absolutely required. You should find this goal easy to accomplish because when working with React components, you want to limit the number of components that have state.

State Within the Component Tree

In many React applications, it is possible to group all state data in the root component. State data can be passed down the component tree via properties, and data can be passed back up the tree to the root via two-way function binding. The result is that all of the state for your entire application exists in one place. This is often referred to as having a "single source of truth."

Color Organizer App Overview

The color organizer allows users to add, name, rate, and remove colors in their customized lists. The entire state of the color organizer can be represented with a single array:

```
{
```

colors: [

{

"id": "0175d1f0-a8c6-41bf-8d02-df5734d829a4",

"title": "ocean at dusk",

"color": "#00c4e2",

"rating": 5

},

{

"id": "83c7ba2f-7392-4d7d-9e23-35adbe186046",

"title": "lawn",

"color": "#26ac56",

"rating": 3

},

{

"id": "a11e3995-b0bd-4d58-8c48-5e49ae7f7f23",

"title": "bright red",

"color": "#ff0000",

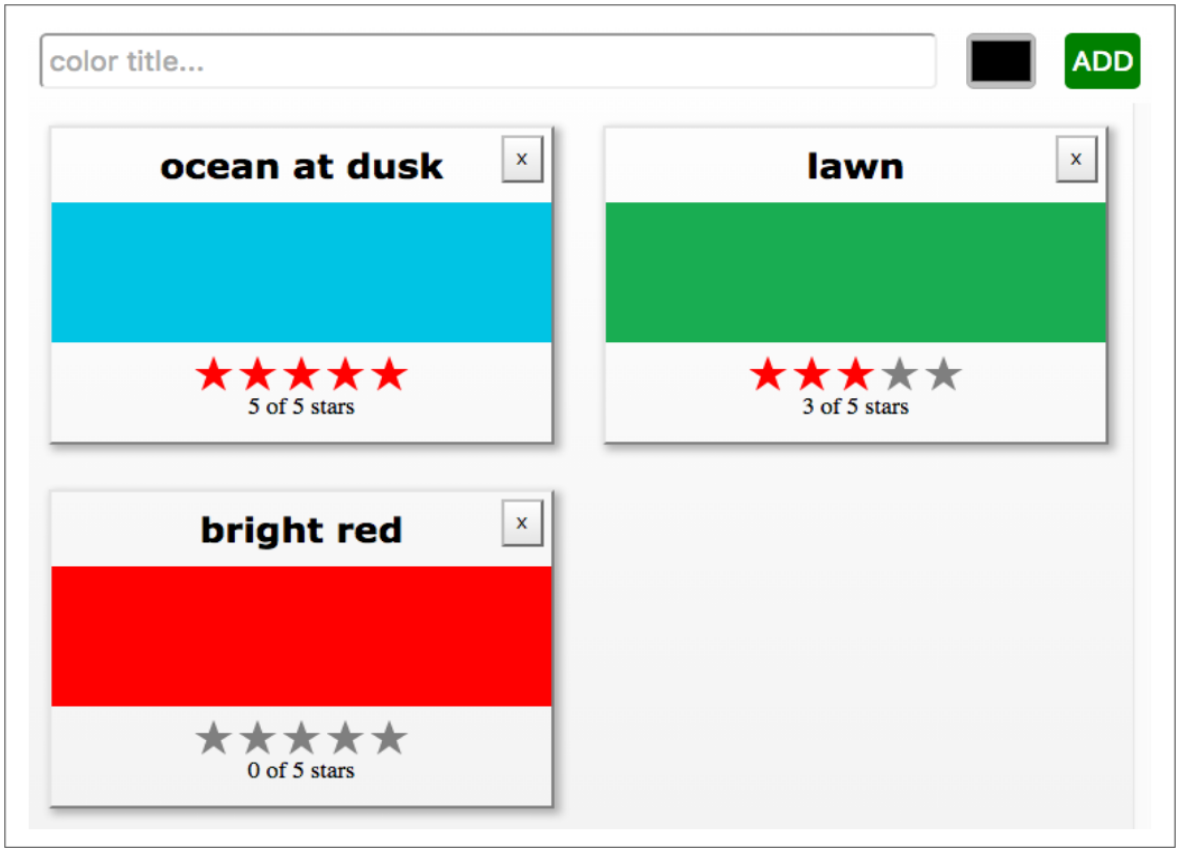
"rating": 0

}

]

}

The array tells us that we need to display three colors: ocean at dusk, lawn, and bright red .It gives us the colors' hex values and the current rating for each color in the display. It also provides a way to uniquely identify each color.



This state data will drive our application. It will be used to construct the UI every time this object changes. When users add or remove colors, they will be added to or removed from this array. When users rate colors, their ratings will change in the array.

Passing Properties Down the Component Tree

We created a StarRating component that saved the rating in the state. In the color organizer, the rating is stored in each color object. It makes more sense to treat the StarRating as a presentational component5 and declare it as a stateless functional component. Presentational components are only concerned with how things look in the application. They only render DOM elements or other presentational components. All data is sent to these components via properties and passed out of these components via callback functions.

In order to make the StarRating component purely presentational, we need to remove state. Presentational components only use props. Since we are removing state from this component, when a user changes the rating, that data will be passed out of this component via a callback function:

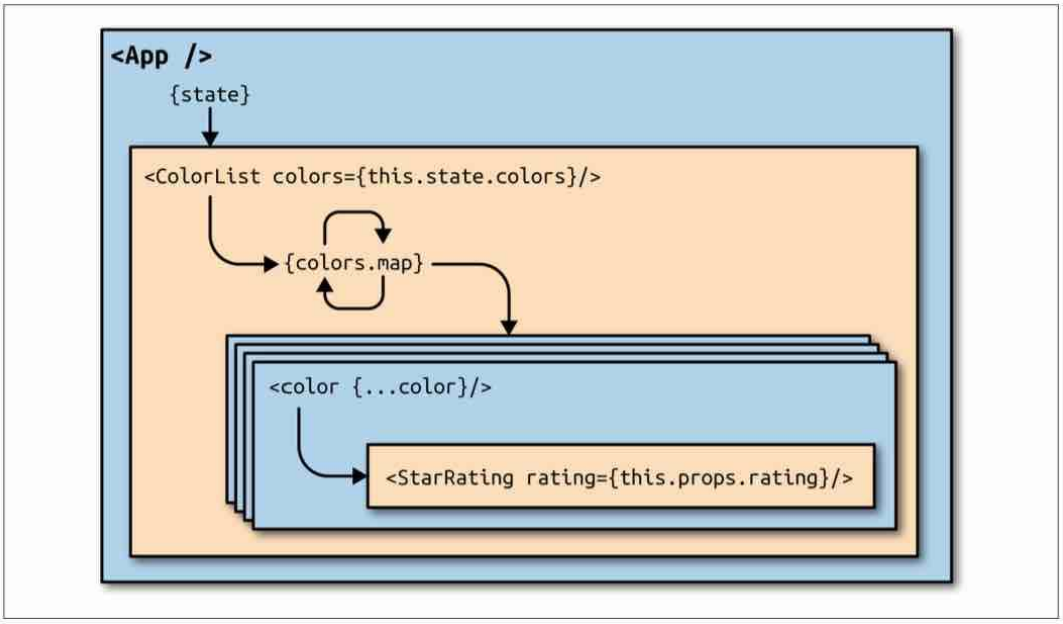
```
const StarRating = ({starsSelected=0, totalStars=5, onRate=f=>f}) =>

<div className="star-rating">

{[...Array(totalStars)].map((n, i) =>

<Star key={i}

selected={i<starsSelected}

onClick={() => onRate(i+1)}/>
```

```
<p>{starsSelected} of {totalStars} stars</p>

)}

</div>
```

First, starsSelected is no longer a state variable; it is a property. Second, an onRate callback property has been added to this component. Instead of calling setState when the user changes the rating, this component now invokes onRate and sends the rating as an argument.

Restricting state to a single location, the root component, means that all of the data must be passed down to child components as properties (Figure).In the color organizer, state consists of an array of colors that is declared in the App component. Those colors are passed down to the ColorList component as a property:

```
class App extends Component {

constructor(props) {

super(props)

this.state = {

colors: []

}

}

render() {

const { colors } = this.state

return (

<div className="app">

<AddColorForm />

<ColorList colors={colors} />

</div>

)

}

}
```



State is passed from the App component to child components as properties

Initially the colors array is empty, so the ColorList component will display a message instead of each color. When there are colors in the array, data for each individual color is passed to the Color component as properties:

```
const ColorList = ({ colors=[] }) =>
<div className="color-list">
{(colors.length === 0) ?
<p>No Colors Listed. (Add a Color)</p> :
colors.map(color =>
<Color key={color.id} {...color} />
)
}
</div>
```

Now the Color component can display the color's title and hex value and pass the color's rating down to the StarRating component as a property:

```
const Color = ({ title, color, rating=0 }) =>
<section className="color">
<h1>{title}</h1>
<div className="color"
style={{ backgroundColor: color }}>
</div>
<div>
```

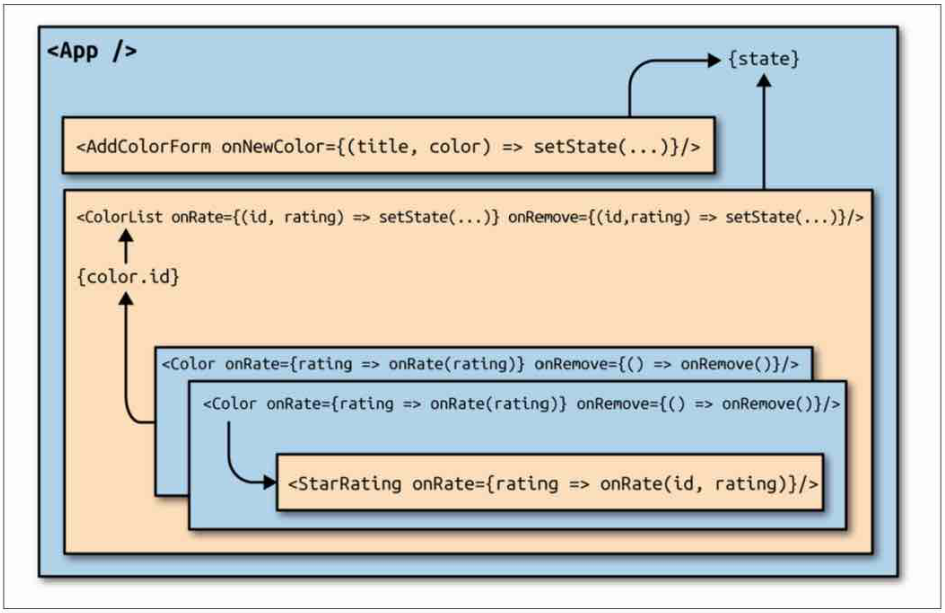
</section>

<StarRating starsSelected={rating} />

The number of starsSelected in the star rating comes from each color's rating. All of the state data for every color has been passed down the tree to child components as properties. When there is a change to the data in the root component, React will change the UI as efficiently as possible to reflect the new state.

Passing Data Back Up the Component Tree

State in the color organizer can only be updated by calling setState from the App component. If users initiate any changes from the UI, their input will need to be passed back up the component tree to the App component in order to update the state This can be accomplished through the use of callback function properties.



Passing data back up to the root component when there are UI events

In order to add new colors, we need a way to uniquely identify each color. This identifier will be used to locate colors within the state array. We can use the uuid library to

create absolutely unique IDs:

npm install uuid --save

All new colors will be added to the color organizer from the AddColorForm component .That component has an optional callback function property called onNewColor. When the user adds a new color and submits the form, the onNewColor callback function is invoked with the new title and color hex value obtained from the user:

```
import { Component } from 'react'
import { v4 } from 'uuid'
import AddColorForm from './AddColorForm'
import ColorList from './ColorList'
export class App extends Component {
```

```
constructor(props) {
super(props)
this.state = {
colors: []
}
this.addColor = this.addColor.bind(this)
}
addColor(title, color) {
const colors = [
...this.state.colors,
{
id: v4(),
title,
color,
rating: 0
]
}
this.setState({colors})
}
render() {
const { addColor } = this
const { colors } = this.state
return (
<div className="app">
<AddColorForm onNewColor={addColor} />
<ColorList colors={colors} />
</div>
)
}
}
```

All new colors can be added from the addColor method in the App component. This function is bound to the component in the constructor, which means that it has access to this.state and this.setState.